

LAPORAN PRATIKUM IX
PEMOGRAMAN BERBASIS OBJEK

“Thread (Multithreading)”



Disusun oleh :

Naufal Dira Agustian - 22115330011

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2025

1. PENDAHULUAN

Pada awalnya, komputer hanya dapat menjalankan satu program pada satu waktu yang menyebabkan ketidakefisienan waktu, namun seiring berkembangnya sistem operasi, muncul kemampuan *multitasking* yang memungkinkan eksekusi beberapa pekerjaan secara bersamaan . Salah satu teknik utamanya adalah *multithreading*, yaitu menjalankan beberapa *thread* atau sub-proses ringan dalam satu proses dengan berbagi alamat memori yang sama untuk mempercepat kinerja aplikasi dan memanfaatkan sumber daya CPU secara optimal . Oleh karena itu, praktikum ini bertujuan agar mahasiswa dapat memahami konsep dasar dan siklus hidup *thread* , mampu membuat *thread* menggunakan berbagai metode seperti *class Thread*, *interface Runnable*, maupun *Lambda Expression* , serta mampu mengimplementasikan konsep *multithreading* tersebut ke dalam aplikasi berbasis GUI untuk mensimulasikan proses, seperti pengunduhan file, secara paralel dan responsif.

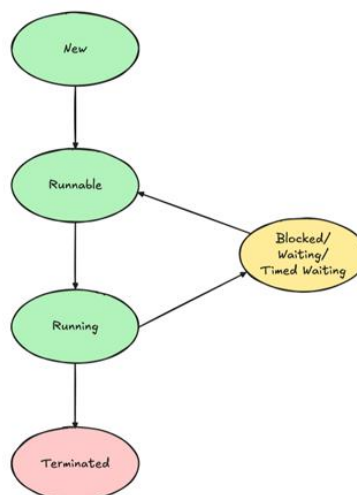
2. LANDASAN TEORI

2.1 Pengertian Thread

Thread dapat diartikan sebagai sub-proses yang ringan (*lightweight*), di mana biaya komunikasi antar *thread* rendah dan perpindahan eksekusi berlangsung cepat. Dalam Java, setiap aplikasi minimal memiliki satu *thread* utama yang disebut main thread .

2.2 Siklus Hidup Thread (Thread Life Cycle)

Sebuah *thread* memiliki lima status utama dalam siklus hidupnya, yaitu: *New*, *Runnable*, *Running*, *Non-Runnable* (*Blocked/Waiting*), dan *Terminated*. Berikut adalah ilustrasi siklus hidup *thread*:



(Ambil gambar diagram lingkaran proses thread dari Halaman 2 modul dan tempel di sini)

Penjelasan status:

1. **New:** *Thread* baru dibuat tetapi belum dimulai (belum dipanggil method `start()`).
2. **Runnable:** *Thread* siap dijalankan dan menunggu alokasi waktu dari CPU.
3. **Running:** *Thread* sedang mengeksekusi instruksi di dalam method `run()`.
4. **Blocked/Waiting:** *Thread* sementara tidak aktif karena menunggu sumber daya atau waktu tertentu (misalnya menggunakan `sleep()`).
5. **Terminated:** *Thread* telah selesai melakukan tugasnya.

2.3 Pembuatan Thread

Di Java, *thread* dapat dibuat dengan tiga cara utama:

1. **Extends Class Thread:** Membuat kelas turunan dari `Thread` dan meng-override method `run()`.
2. **Implements Interface Runnable:** Mengimplementasikan interface `Runnable` dan mendefinisikan method `run()`.
3. **Lambda Expression:** Fitur Java 8+ yang menyederhanakan penulisan kode untuk *Runnable*.

3. HASIL DAN PEMBAHASAN

3.1 Tugas 2: Simulasi Download dengan Lambda Expression

Pada tugas ini, dilakukan simulasi proses pengunduhan 3 file secara paralel. Kode program dimodifikasi dari latihan sebelumnya untuk menggunakan *Lambda Expression* agar lebih ringkas sesuai instruksi modul.

Kode Program (ConsoleDownloadLambda.java):

```
package Pratikum9;

public class ConsoleDownloadLambda {

    public static void main(String[] args) {

        // Implementasi Runnable menggunakan Lambda Expression

        Runnable downloadTask = () -> {

            String name = Thread.currentThread().getName();

            for (int i = 10; i <= 100; i += 10) {

                System.out.println(name + " progress: " + i + "%");

                try {

                    Thread.sleep(500); // Simulasi delay download
```

```

    } catch (InterruptedException e) {

        e.printStackTrace();

    }

}

System.out.println(name + " selesai diunduh!");

};

// Membuat 3 Thread yang berjalan paralel

Thread t1 = new Thread(downloadTask, "File-1");

Thread t2 = new Thread(downloadTask, "File-2");

Thread t3 = new Thread(downloadTask, "File-3");

t1.start();

t2.start();

t3.start();

}

}

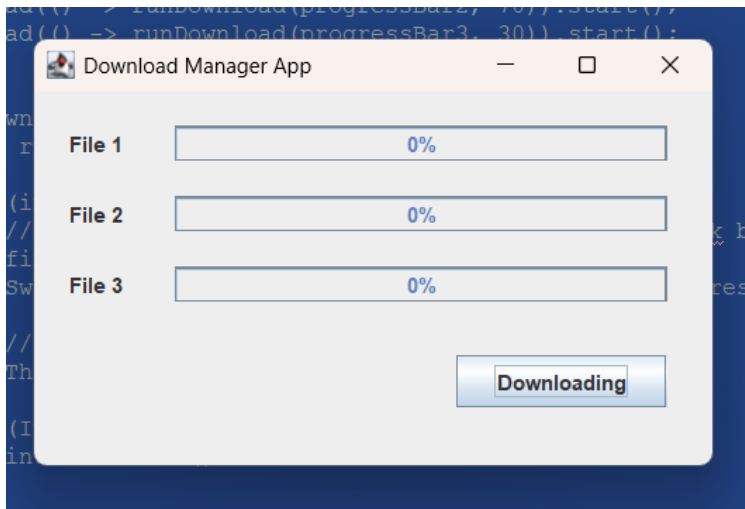
```

Analisis: Pada kode di atas, implementasi Runnable ditulis menggunakan sintaks () -> { ... }. Di dalam blok tersebut, terdapat perulangan for dari 10 hingga 100 untuk mensimulasikan progres. Method Thread.sleep(500) digunakan untuk memberi jeda waktu, seolah-olah proses unduhan sedang berlangsung. Karena ketiga *thread* (t1, t2, t3) dijalankan dengan start(), output di konsol muncul secara bergantian, menandakan proses berjalan paralel.

3.2 Tugas 3: GUI Download Manager App

Pada tugas ini, dibuat aplikasi GUI menggunakan Java Swing yang menampilkan *progress bar* untuk 3 file yang diunduh bersamaan. Tampilan aplikasi dibuat menyerupai desain referensi pada modul.

Gambar 2. Desain GUI Download Manager



Kode Program (DownloadManagerApp.java):

```
package Pratikum9;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;

public class DownloadManagerApp extends JFrame {

    // Komponen GUI

    private JProgressBar progressBar1;
    private JProgressBar progressBar2;
    private JProgressBar progressBar3;
    private JButton btnStart;

    public DownloadManagerApp() {

        // Setup Window (JFrame)
```

```
setTitle("Download Manager App");

setSize(400, 250);

setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

setLocationRelativeTo(null); // Posisi di tengah layar

setLayout(null); // Menggunakan Absolute Layout


// --- Label File 1 ---

JLabel lblFile1 = new JLabel("File 1");

lblFile1.setBounds(20, 20, 50, 20);

add(lblFile1);


// Progress Bar 1

progressBar1 = new JProgressBar(0, 100);

progressBar1.setBounds(80, 20, 280, 20);

progressBar1.setStringPainted(true);

progressBar1.setForeground(new Color(51, 153, 255));

add(progressBar1);


// --- Label File 2 ---

JLabel lblFile2 = new JLabel("File 2");

lblFile2.setBounds(20, 60, 50, 20);

add(lblFile2);


// Progress Bar 2

progressBar2 = new JProgressBar(0, 100);
```

```
progressBar2.setBounds(80, 60, 280, 20);

progressBar2.setStringPainted(true);

progressBar2.setForeground(new Color(51, 153, 255));

add(progressBar2);


// --- Label File 3 ---

JLabel lblFile3 = new JLabel("File 3");

lblFile3.setBounds(20, 100, 50, 20);

add(lblFile3);


// Progress Bar 3

progressBar3 = new JProgressBar(0, 100);

progressBar3.setBounds(80, 100, 280, 20);

progressBar3.setStringPainted(true);

progressBar3.setForeground(new Color(51, 153, 255));

add(progressBar3);


// --- Tombol Downloading ---

// INI BAGIAN PENTING YANG SEBELUMNYA HILANG/NULL:

btnStart = new JButton("Downloading");

btnStart.setBounds(240, 150, 120, 30);

add(btnStart);


// Action Listener (Baru boleh dipanggil setelah btnStart = new JButton...)

btnStart.addActionListener((ActionEvent e) -> {
```

```

        startDownload();

    });

}

// Method untuk memulai proses download secara paralel

private void startDownload() {

    // Non-aktifkan tombol agar tidak diklik berkali-kali

    btnStart.setEnabled(false);

    btnStart.setText("Process...");


    // Reset progress bar

    progressBar1.setValue(0);

    progressBar2.setValue(0);

    progressBar3.setValue(0);


    // Jalankan Thread untuk masing-masing progress bar

    new Thread(() -> runDownload(progressBar1, 50)).start();

    new Thread(() -> runDownload(progressBar2, 70)).start();

    new Thread(() -> runDownload(progressBar3, 30)).start();

}


// Logika download simulasi

private void runDownload(JProgressBar bar, int sleepTime) {

    try {

        for (int i = 0; i <= 100; i++) {

```



```

        // Update nilai progress bar (harus dalam EDT untuk best practice, tapi ini
aman untuk simulasi simple)

        final int progress = i;

        SwingUtilities.invokeLater() -> bar.setValue(progress));

        // Simulasi delay waktu download

        Thread.sleep(sleepTime);

    }

} catch (InterruptedException e) {

    e.printStackTrace();

}

}

public static void main(String[] args) {

    SwingUtilities.invokeLater() -> {

        DownloadManagerApp frame = new DownloadManagerApp();

        frame.setVisible(true);

    });

}

}

```

Analisis:

1. **Komponen GUI:** Aplikasi menggunakan JFrame sebagai wadah utama, JProgressBar untuk menampilkan persentase unduhan secara visual, dan JButton untuk memicu aksi.
2. **Multithreading:** Saat tombol "Downloading" ditekan, aplikasi membuat 3 *thread* baru (bukan menjalankan logika di *Main Thread*).

Java

```
new Thread(() -> runDownload(progressBar1, 50)).start();
```

Hal ini sangat penting agar antarmuka pengguna (GUI) tidak *freeze* (macet) saat proses *looping* dan sleep berjalan.

3. **Simulasi Kecepatan:** Parameter waktu pada `Thread.sleep(time)` dibedakan untuk setiap file (50ms, 70ms, 30ms) agar simulasi terlihat realistis di mana setiap file selesai dalam waktu yang berbeda.

4. PENUTUP

4.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Penggunaan **Thread** memungkinkan aplikasi untuk melakukan *multitasking*, yaitu mengerjakan beberapa tugas berat secara bersamaan tanpa harus menunggu satu tugas selesai terlebih dahulu.
2. **Lambda Expression** pada Java sangat membantu dalam menyederhanakan penulisan kode *thread*, membuat kode lebih bersih dan mudah dibaca dibandingkan menggunakan *Anonymous Inner Class*.
3. Dalam pengembangan aplikasi GUI, pemisahan proses logika (seperti unduhan) dari *Event Dispatch Thread* (EDT) menggunakan *thread* terpisah adalah hal wajib agar aplikasi tetap responsif terhadap input pengguna.